En esta práctica aprenderemos los principios básicos de despliegue de aplicaciones en contenedores Docker.

1ª parte - Despliegue de un contenedor Docker

Tarea 0. Instala Docker Desktop:

Instalación de Docker Desktop:

- **Para Windows: https://docs.docker.com/desktop/windows/install/**

  Recuerda que en Windows, la instalación te pedirá que instales, previamente, el kernel de Linux (WSL2): https://docs.microsoft.com/es-es/windows/wsl/install-manual#step-4---download-the-linux-kernel-update-package

- **Para Mac: https://docs.docker.com/desktop/mac/install/**

Tarea 1 (2 puntos). a)Prepara una carpeta *holaMundo* en tu sistema de ficheros local que contenga el siguiente contenido:

-Una carpeta *src* con un simple *index.php* en su interior como el siguiente:-Un fichero *Dockerfile* (sin extensión), con el siguiente contenido:

b) Desde el terminal de línea de comandos, ubícate en la raíz del proyecto y ejecuta el siguiente comando para compilar una nueva imagen:

docker build -t holamundo .

```
src
index.php
Dockerfile
```

```
src > index.php
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta http-equiv="X-UA-Compatible" content="IE=edge">
6  <meta name="viewport" content="width=device-width, initial-scale=1.0">
7  <title>Hola Mundo</title>
8  </head>
9  <body>
10 <?php
11 echo "Hola Mundo desde un contenedor Docker";
12 ?>
13 </body>
14 </html>
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   powershell

PS C:\docker-php-apache-mysql\src\holaMundo> docker build -t holamundo .
 => => extracting sha256:54695fdb10a725b6bccb0ce6d54c677c64f97f296e1bfa4dae883a2644  0.6s
 => => extracting sha256:500ca11be45fbc9e5136a3bf94561be4a89c08897a0eca0dfe058432ea  0.1s
 => => extracting sha256:86b2805859cff8a91f471267a71e24de74994e3f6201d88ff64d77f07b  3.9s
 => => extracting sha256:c61685fa4f4f0aaae0d11ad60af9bc4ba90e170b51e3ab6f70e5a637d3  0.1s
 => => extracting sha256:0bf989f9dbbb72db9b0006970a86b1fdd2f1610ef9b79674ce5fa6cd64  0.1s
 => => extracting sha256:01848ea209b526414bdc69c038ed8d867e7af64082f7a468a2d7d7cec7  0.1s
 => [2/2] COPY src/ /var/www/html                                                   0.9s
 => exporting to image                                                              0.9s
 => => exporting layers                                                             0.3s
 => => exporting manifest sha256:694867dc9b9eeeebe8d3b147291429632bfb96551104fe5272  0.0s
 => => exporting config sha256:36af61e6b08560d6cf08203e51ff514ea0c0f791efed9c56a250  0.0s
 => => exporting attestation manifest sha256:df9ff60d5947ec77b7e502e6f216fcc38f9c60  0.1s
 => => exporting manifest list sha256:163a89e13b402c2d9fdd1612ed1a7780394097e19fbfd  0.0s
 => => naming to docker.io/library/holamundo:latest                                 0.0s
 => => unpacking to docker.io/library/holamundo:latest                              0.1s
PS C:\docker-php-apache-mysql\src\holaMundo>
```

Observa cómo Docker descarga la imagen base (capa *php:7.1-apache*), y compila a partir de ella una nueva imagen con las instrucciones dispuestas en el *Dockerfile*. La imagen se llamará *holamundo*.

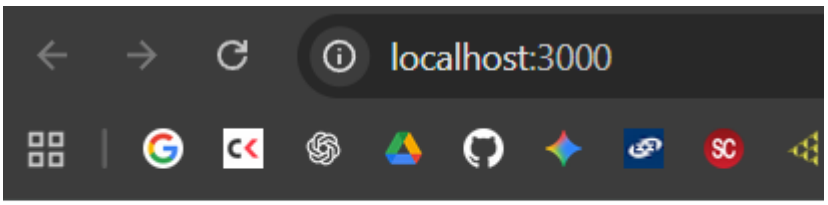
c) Arranca un nuevo contenedor Docker que ejecute la imagen que acabas de crear:

docker run -dp 3000:80 holamundo

```
PS C:\docker-php-apache-mysql\src\holaMundo> docker run -dp 3000:80 holamundo
4214ee2234c49eadac7c0cb199e498f3f9802fca71444e63e18286b1ae1541d5
PS C:\docker-php-apache-mysql\src\holaMundo>
```

d) Visualiza la salida de tu aplicación en

http://localhost:3000/



Hola Mundo desde un contenedor Docker

e) Analizemos lo que acaba de ocurrir. Para ello, explica:

- El comando FROM define la imagen base con Linux, Apache y PHP 7.1. COPY traslada los archivos de la carpeta local src al directorio de trabajo del servidor web en el contenedor. EXPOSE informa que el servicio utiliza el puerto 80. Respecto al arranque, docker run inicia el contenedor. La opción -d permite que corra en segundo plano sin bloquear la terminal. El parámetro -p 3000:80 vincula el puerto 3000 de tu equipo con el 80 del contenedor para permitir el acceso web. Finalmente, holamundo indica qué imagen debe usarse para crear dicho contenedor.

f) Observa que puedes acceder a la terminal de la sesión de Linux que acabas de desplegar a través del panel de control de Docker Desktop.

En Docker Desktop → Containers → tu contenedor → botón Terminal

Verás una sesión Linux dentro del contenedor. Puedes navegar por el sistema de ficheros y ver los archivos copiados en `/var/www/html`.

Tarea 2 (1 punto) a) Utiliza el comando *docker ps* para listar los contenedores activos. Observa que a cada contenedor se le asigna un ID y un nombre aleatorio (un nombre curioso IMHO).

```
PS C:\docker-php-apache-mysql\src\holaMundo> docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS         PORTS
                                        NAMES
4214ee2234c4   holamundo   "docker-php-entrypoi…"   8 minutes ago   Up 8 minutes   0.0.0.0
:3000->80/tcp, [::]:3000->80/tcp   elegant_chaum
PS C:\docker-php-apache-mysql\src\holaMundo>
```

b) Para el contenedor y bórralo

de memoria: ***docker stop***

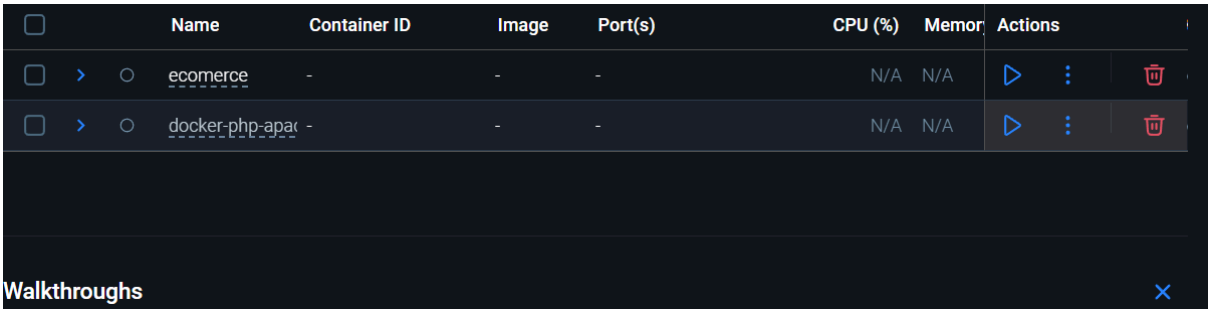
<nombre-contenedor>

docker rm

<nombre-contenedor>

```
PS C:\docker-php-apache-mysql\src\holaMundo> docker stop 4214ee2234c4
4214ee2234c4
PS C:\docker-php-apache-mysql\src\holaMundo> docker rm 4214ee2234c4
4214ee2234c4
PS C:\docker-php-apache-mysql\src\holaMundo>
```

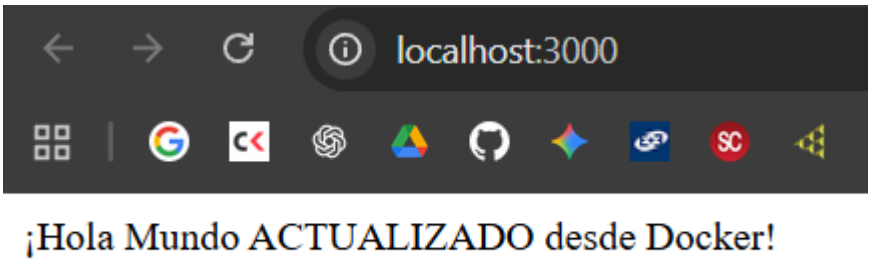
c) Verás que desaparece del panel de control de Docker.



b) Actualiza el código de tu aplicación: por ejemplo, cambia el valor del *echo*.

```
<?php
echo "¡Hola Mundo ACTUALIZADO desde Docker!";
?>
</body>
```

c) Recompila la imagen y vuelve a arrancar el contenedor. Observa los cambios en el navegador.



Tarea 3 (2 puntos) – Ahora vamos a montar el código de la aplicación como un volumen externo al contenedor (*bind mount*), para que no haya que recompilar cada vez los cambios que se realizan en el código.

a) Borra el contenedor que acabas de crear.

```
PS C:\docker-php-apache-mysql\src\holaMundo> docker stop 9bb81226f2d6a1c9071ee9b682d5c3086
56123a2b6e3e949c14400e7770e6ae6
9bb81226f2d6a1c9071ee9b682d5c308656123a2b6e3e949c14400e7770e6ae6
PS C:\docker-php-apache-mysql\src\holaMundo> docker rm 9bb81226f2d6a1c9071ee9b682d5c308656
123a2b6e3e949c14400e7770e6ae6
9bb81226f2d6a1c9071ee9b682d5c308656123a2b6e3e949c14400e7770e6ae6
PS C:\docker-php-apache-mysql\src\holaMundo>
```

b) Comenta la línea *COPY...* del fichero Dockerfile para que no se copie el código de la aplicación a la imagen compilada.

```
Dockerfile > ...
1   FROM php:7.1-apache
2   # COPY src/ /var/www/html
3   EXPOSE 80
```

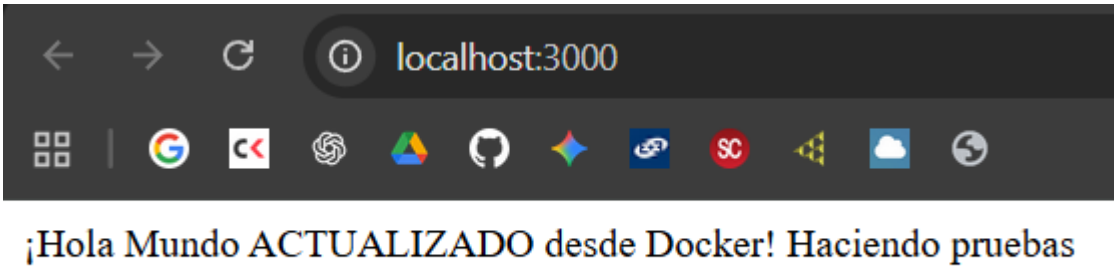
c) Recompila la imagen.

```
PS C:\docker-php-apache-mysql\src\holaMundo> docker build -t holamundo .
 => [internal] load build definition from Dockerfile                                0.3
 => => transferring dockerfile: 95B                                                 0.2
 => [internal] load metadata for docker.io/library/php:7.1-apache                   3.4
 => [internal] load .dockerignore                                                   0.0
 => => transferring context: 2B                                                     0.0
 => CACHED [1/1] FROM docker.io/library/php:7.1-apache@sha256:70eefcf4372b279101259  0.1
 => => resolve docker.io/library/php:7.1-apache@sha256:70eefcf4372b279101259e905996  0.1
```

d) Ahora <u>arranca el contenedor con la opcion -v</u>, para que el código se monte como un volumen externo a la imagen:

```
PS C:\docker-php-apache-mysql\src\holaMundo> docker run -dp 3000:80 -v ${PWD}/src:/var/www
/html holamundo
3c0358c146afd5d9f379ef675543b1b0c351583e3e73595bfc95ffd7c7e60bbc
PS C:\docker-php-apache-mysql\src\holaMundo>
```

e) Comprueba que ahora los cambios en el código se refrescan automáticamente en el *front-end*.



Tarea 4 (2 puntos) – A continuación, montaremos un CRUD completo, con base de datos incluida, conectando dos contenedores en red.

<u>-Operaciones previas:</u>

a) Primero, actualiza el contenido de la carpeta *src* de tu proyecto con el contenido de la carpeta *phpcrud* adjunta.

b) Borra los contenedores creados hasta ahora.

c) Crea una nueva red en Docker. La utilizaremos para conectar los dos contenedores entre sí:

***docker network create crud*.**

(Puedes listar las redes existentes con *docker network ls*).

```
PS D:\Users\alumn\Documents\DAW 2\Desplegament d'aplicacions web\M8AC32> docker network create crud
4d99b8c2837a896cfd258e00e2ce889016aef8e2fc5ea2edd50f44ba7f49669a
```

-Arranque del contenedor de MySQL:

d) Ahora arrancaremos un contenedor con MySQL a partir de una imagen oficial del Docker Hub (marcada en azul):

***docker run -d ***
***--network crud --network-alias mysql ***
***-v crud-mysql-data:/var/lib/mysql ***
***-e MYSQL_ROOT_PASSWORD=1234 -e MYSQL_DATABASE=crud ***
mysql:5.7

Atención, se trata de un único comando.
Solo se divide en líneas para mejorar la legibilidad.

-Observa que:

-En este caso no hace falta compilar previamente la imagen: la usamos tal como está (*mysql:5.7)*, descargada directamente desde el repositorio de imágenes oficiales, Docker Hub.

-Lanzamos el contenedor asociado a una red (*crud*) y con un alias en esa red (*mysql*), para poder ser identificado fácilmente.

-Montamos en el contenedor un volumen externo. El punto de montaje en el sistema de ficheros linux es */var/lib/mysql*, y el nombre del volumen externo es *crud-mysql-data*. En este caso el volumen externo no es ningún directorio de nuestro equipo amfitrión, sino un volumen que Docker se encarga de gestionar. Puedes inspeccionarlo en el panel de control de Docker Desktop.

```
PS D:\Users\alumn\Documents\DAW 2\Desplegament d'aplicacions web\M8AC32\phpcrud\phpcrud\src> docker run -d --network crud --netw
ork-alias mysql -v crud-mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=1234 -e MYSQL_DATABASE=crud mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
df9a4d85569b: Download complete
20e4dcae4c69: Downloading [==================================================>]  48.57MB/50.5MB
ae71319cb779: Download complete
e9f03a1c24ce: Download complete
43d05e938198: Downloading [=====================================>]  46.14MB/56.29MB
68c3898c2015: Download complete
ffc89e9dfd88: Download complete
1c56c3d4ce74: Download complete
```

e) Des de Docker Desktop, [(o desde el propio VSCode, si tienes instalada la extensión Docker de Microsoft),](#) abre el terminal del contenedor con MySQL. Entra en el cliente de *mysql* con el usuario *root*. Comprueba que la base de datos *crud* está creada. Selecciónala y ejecuta el contenido del *script crud.sql* que se incluye en el proyecto. Puedes copiar y pegar el contenido directamente al terminal, una vez has entrado en el cliente *mysql*.

```
mysql> USE crud;
Database changed
mysql>
mysql> -- ------------------------------------------------------
mysql>
mysql> --
mysql> -- Table structure for table `people`
mysql> --
mysql>
mysql> CREATE TABLE IF NOT EXISTS `people` (
    ->   `people_id` int(11) NOT NULL AUTO_INCREMENT,
    ->   `first_name` varchar(30) NOT NULL,
    ->   `last_name` varchar(30) NOT NULL,
    ->   `mid_name` varchar(30) NOT NULL,
    ->   `address` varchar(30) NOT NULL,
    ->   `contact` varchar(30) NOT NULL,
    ->   `comment` text NOT NULL,
    ->   PRIMARY KEY (`people_id`)
    -> ) ENGINE=InnoDB  DEFAULT CHARSET=latin1 AUTO_INCREMENT=4 ;
```

f) Usa *docker volume ls* para listar los volúmenes. Observa que Docker ha creado un volumen para el uso del contenedor de MySQL, con el nombre que le hemos asignado en el comando

de arranque del contenedor. Esto hace que sus datos sean persistentes. Anota el nombre del volumen. También puedes inspeccionarlo en Docker Desktop.

```
PS D:\Users\alumn\Documents\DAW 2\Desplegament d'aplicacions web\M8AC32\phpcrud\phpcrud\src> docker volume ls
DRIVER    VOLUME NAME
local     crud-mysql-data
local     docker-php-apache-mysql_db_data
local     ecomerce_db_data
local     exemple_db_data
local     laravel-docker_db_data
PS D:\Users\alumn\Documents\DAW 2\Desplegament d'aplicacions web\M8AC32\phpcrud\phpcrud\src>
```

-Arranque del contenedor de aplicación:

g) **Actualiza el Dokerfile para que se instalen en la imagen los conectores de PHP a MySQL. Observa como completamos la imagen a partir de la capa base *php:7.1-apache*:**

g) **Comprueba que en el archivo *conexion.php* figuran los parámetros de conexión adecuados (host, usuario, contraseña, etc). El *host* debe ser el alias del contenedor de MySQL en la red Docker. Recuerda que no te estarás conectando a tu servidor local de MySQL (si lo tuvieres), sino al contenedor Docker que acabamos de arrancar con una imagen de MySQL.**

connection.php

phpcrud > phpcrud > src > connection.php

```php
<?php
$db = mysqli_connect('mysql', 'root', '1234', 'crud') or
        die ('Unable to connect. Check your connection parameters.');
?>
```

h) **Recompila la imagen de tu proyecto con el nombre *crud*.**

i) **Arranca un contenedor con la imagen que acabas de compilar (en azul), ahora indicando que queremos incluir el contenedor en la red *crud*:**

```
docker run -dp 3000:80 \
 -v "%cd%"/src:/var/www/html \
--network crud \
crud
```

j) **Comprueba que la aplicación se presenta adecuadamente en http://localhost:3000/. Si no es así, deberás depurar los pasos anteriores.**

2ª parte - Despliegue de un contenedor Docker Compose

La aplicación DOCKER COMPOSE permite componer aplicaciones (mono o multi-contenedor) de manera más cómoda. Una APLICACIÓN encapsula distintos SERVICIOS, cada uno de ellos corriendo en un contenedor separado. Generar una aplicación con Docker Compose facilita enormemente el proceso de configuración, compilación y despliegue de una imagen, especialmente en los casos de múltiples contenedores.

Tarea 00 – Instalación de Docker Compose:

Para ello, sigue las instrucciones en https://docs.docker.com/compose/install/.

> Si utilizas la extensión Docker de VSCode, no es necesario instalar separadamente Docker Compose. La extensión implementa los comandos de composición que utilizaremos.

Tarea 4 (2 puntos) – Una aplicación generable con Docker Compose se describe en un fichero *docker-compose.yml*. Este fichero sigue el formato YAML (que se utiliza también en los ficheros de configuración de Hadoop).

En el que se muestra a continuación, se describe una aplicación consistente dos imágenes (que correrán en dos contenedores separados): una imagen customizada (*crud,* que se describe en el *Dockerfile* que ya hemos ido viendo); y una imagen oficial de *mysql*. Nótese que ambas partes de la aplicación se describen como servicios, cada uno de los cuáles lleva asociadas una serie de opciones de configuración (puertos, volúmenes externos, redes a las que está conectado, etc.):

Observa la correlación entre los comandos que usaste para desplegar la aplicación en la actividad anterior y las directivas de configuración del servicio en el *docker-compose.yml.*

a) **¿Dónde se describe la imagen que corresponde al servicio *crud*?**

b) **¿Por qué crees que se utiliza la directiva *image* para describir la imagen del servicio *mysql*, mientras que en el servicio *app* se utiliza la directiva *build*?**

Observa que utilizamos como volumen del contenedor *mysql* un volumen externo. Asegúrate de que utilizas el volumen que inspeccionaste en la Tarea 4.f. Esto permitirá que la aplicación trabaje con una base de datos ya inicializada.

c) **El fichero *docker-compose.yml* debe incluirse en el directorio de proyecto, junto con el fichero *Dockerfile*. Una vez descrita la aplicación en el YAML, ésta puede desplegarse solamente en un paso, con el comando**

docker-compose up -d

d) Muestra la aplicación funcionando en http://localhost:3000

e) ¿Para que se utiliza el la opción *-d* en el comando?

Tarea 5 (1 punto) – a)Puedes replegar la aplicación (esto es, parar su ejecución y eliminarla de memoria), com el comando

docker-compose down

b) Vuelve a desplegarla mediante el comando de la tarea anterior, y comprueba cómo la aplicación aparece de nuevo en el panel de control de Docker Desktop.

c) ¿Se guarda, entre sucesivas ejecuciones de la aplicación, la información introducida en la base de datos? ¿Por qué?

ATENCIÓN: Los contenedores son procesos que consumen memoria. Hay que acostumbrarse a pararlos y eliminarlos cuando ya no se utilicen.

Las imágenes son ficheros que ocupan mucho espacio en disco. Hay que acostumbrarse a borralos cuando no se utilicen. No hay que sufrir por volverlos a necesitar. En los ficheros de configuración (Dockerfile, docker-compose.yml) está toda la información necessaria para volver a levantar los contenedores.

Otra cosa pasa con los volúmenes externos. Estos sólo deben ser eliminados si no queremos persistir la información entre sucesivas ejecuciones de la aplicación.

Puedes gestionar las tres cosas cómodamente desde el panel de control de Docker Desktop.

ANEXO – Otro ejemplo: *docker-compose.yml* para levantar una pila LAMP (Linux + Apache + MySQL + PHP) que sirve Wordpress.